Smart Irrigation System (IoT-based)

Course Name: Microprocessor

Course code: EEC214

Name	ID
Ahmed Bushra Ahmed	20243166
Islam Elsayed Ghonmey	20210536
Abdelrahman Ehab Issa	20200580
Mohamed Tharwat Mohamed	20160772

Abstract

This report presents the design and implementation of a smart irrigation system based on a microcontroller using ESP32. The system aims to automate the irrigation process by monitoring soil moisture levels and controlling water supply accordingly, reducing water wastage and minimizing human intervention.

A soil moisture sensor is used to continuously measure the moisture content in the soil. The collected data is processed by the microcontroller and compared with a predefined threshold value. When the soil is dry, the system activates a relay module to turn on a water pump. Once the soil reaches an adequate moisture level, the system automatically turns off the pump.

The system was tested through practical implementation using low-cost electronic components. The results demonstrate that the proposed system operates efficiently and reliably in maintaining optimal soil conditions. This makes it a cost-effective and scalable solution suitable for smart agriculture applications.

Acknowledgement

First and foremost, I would like to express my deepest gratitude to my parents for their endless love, continuous support, and encouragement throughout my academic journey. Their motivation has always been my greatest strength.

I would also like to sincerely thank **Dr. Mohamed Torad** for his valuable guidance, support, and supervision during the completion of this project. His advice and feedback were essential in improving the quality of this work.

Introduction

In recent years, the demand for efficient water management systems in agriculture has increased significantly due to water scarcity and the need for sustainable farming practices. Traditional irrigation methods often rely on manual operation, which can lead to water wastage and inefficient resource utilization.

This project presents a smart irrigation system based on a microcontroller using ESP32. The system is designed to automatically monitor soil moisture levels and control the irrigation process without human intervention. By using sensors and automation, the system ensures that water is supplied only when needed, helping to maintain optimal soil conditions.

The main goal of this project is to develop a low-cost, efficient, and reliable solution that can be used in smart agriculture applications to improve water usage efficiency and reduce manual effort.

Components and Tools

Components	Description
ESP32	Main microcontroller used to control the system and read sensor data
Soil Moisture Sensor	Measures the moisture level of the soil
DHT11	Measures temperature and humidity in the environment
Relay Module	Acts as a switch to control the water pump
Water Pump	Used for irrigation
BC107 Transistor	Used as a switching component to drive or control circuits
Breadboard	Used for assembling and testing the circuit
Jumper Wires	Used for connecting the electronic components
WiFi Module (Built-in ESP32)	Enables internet connectivity for remote control

Software Used

Arduino IDE

Arduino IDE is the main programming environment used to write, compile, and upload the code to the ESP32. It allows easy development using C/C++ language and provides a Serial Monitor to display sensor readings and debug the system during testing.

Proteus Design Suite

Proteus is used to design and simulate the electronic circuit before building the physical prototype. It helps in testing the system behavior virtually, including sensor readings, microcontroller responses, and relay operation, which reduces errors during hardware implementation.

Web Server (ESP32 HTTP Server)

The system includes an embedded web server running on the ESP32 that allows users to monitor sensor data (soil moisture, temperature, humidity) and control the water pump remotely via HTTP requests using a browser or mobile device

Methodology

The system is developed using an ESP32 as the main controller. It operates as an IoT-based smart irrigation system that integrates automatic control, remote monitoring, and wireless communication.

At the beginning of operation, the system continuously reads data from a soil moisture sensor to determine the water content of the soil. In parallel, environmental conditions such as temperature and humidity are measured using a DHT11.

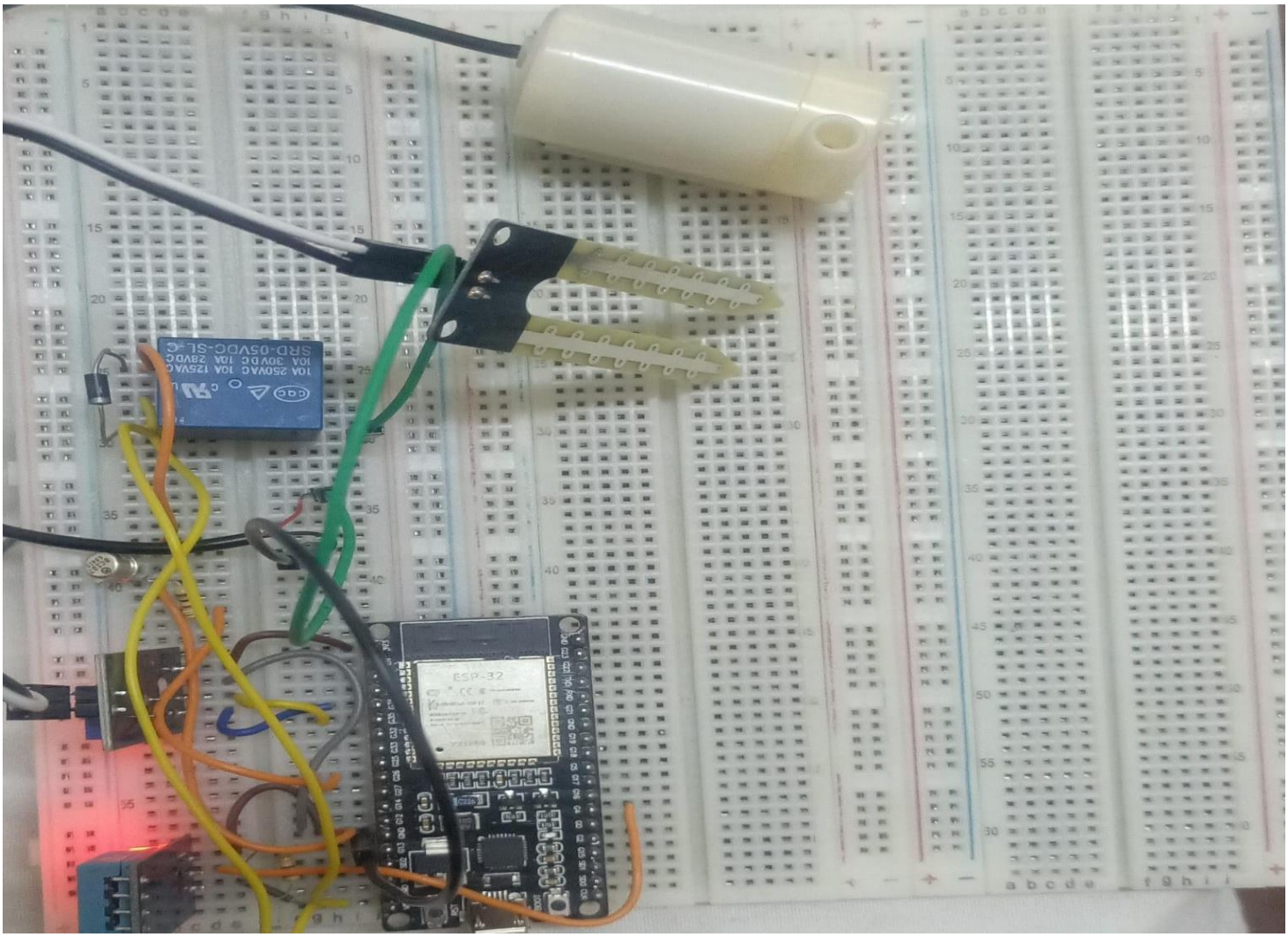
The microcontroller processes all sensor readings and compares the soil moisture value with a predefined threshold. If the soil is dry, the system automatically activates a relay module to turn ON the water pump. When the soil reaches an adequate moisture level, the pump is automatically turned OFF.

In addition to automatic irrigation, the system supports manual control through WiFi and Bluetooth Low Energy (BLE). Users can remotely monitor sensor data and control the pump via a built-in web server or BLE commands.

The system also stores important operational data such as usage statistics in non-volatile memory to maintain information even after power loss. This ensures reliable and continuous operation of the irrigation system.

Hardware Implementation

The figure below shows the hardware implementation of the smart irrigation system. It includes the ESP32, soil moisture sensor, DHT11, relay module, and water pump connected together to form the complete system prototype.



Mobile Application Interface

The figures below show the mobile application interface used to monitor sensor data and control the irrigation system remotely via WiFi/BLE.

Results

The smart irrigation system was successfully implemented and tested using an ESP32 along with a soil moisture sensor, relay module, water pump, and a DHT11.

The results demonstrated that the system operates efficiently in both automatic and manual modes. In automatic mode, the system accurately detects soil moisture levels and controls the water pump accordingly; the pump turns ON when the soil is dry and turns OFF when sufficient moisture is reached.

In addition to hardware performance, the system successfully provides real-time monitoring and control through a mobile application interface and a web-based dashboard. Sensor readings such as soil moisture, temperature, and humidity are displayed live on the application.

Furthermore, the system allows remote control of the water pump using WiFi and Bluetooth Low Energy (BLE), which enhances usability and flexibility. The communication between the application and the microcontroller was stable and responsive during testing.

Overall, the system achieved reliable performance, reduced manual intervention, and successfully fulfilled the objective of developing a smart and efficient IoT-based irrigation system.

System Implementation Code

```
#include <WiFi.h>
#include <WebServer.h>
#include <DHT.h>
#include <Preferences.h>
#include <ESPmDNS.h>
#include <ArduinoJson.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>
#include <HTTPClient.h>

#define PUMP_PIN  26
#define SOIL_PIN  34
#define DHT_PIN   4
#define DHT_TYPE  DHT11

#define THINGSPEAK_API_KEY
"PLUWZ7GWBR09K5IS"
#define THINGSPEAK_URL
"http://api.thingspeak.com/update"

#define SERVICE_UUID        "12345678-1234-1234-
1234-123456789abc"
#define CHARACTERISTIC_UUID "abcdefab-cdef-
abcd-efab-cdefabcdefab"

DHT dht(DHT_PIN, DHT_TYPE);
WebServer server(80);
Preferences prefs;
BLEServer* pServer = NULL;
BLECharacteristic* pCharacteristic = NULL;
bool bleConnected = false;

float totalWaterTime = 0;
```

```
int waterCount = 0;
String deviceName = "SmartDrop-1";
String secretKey = "";
String wifiSSID = "";
String wifiPass = "";
bool configured = false;
bool pumpState = false;

unsigned long lastAutoCheck   = 0;
unsigned long lastThingSpeak  = 0;

void sendToThingSpeak(float temp, float humidity, int
soil, bool pump) {
if (WiFi.status() != WL_CONNECTED) return;
if (millis() - lastThingSpeak < 15000) return;
lastThingSpeak = millis();

HTTPClient http;
String url = String(THINGSPEAK_URL) +
"?api_key=" + THINGSPEAK_API_KEY +
"&field1=" + String(temp, 1) +
"&field2=" + String(humidity, 1) +
"&field3=" + String(soil) +
"&field4=" + String(pump ? 1 : 0);

http.begin(url);
http.GET();
http.end();
}

class MyServerCallbacks : public BLEServerCallbacks
{
void onConnect(BLEServer* pServer)    {
bleConnected = true; }
void onDisconnect(BLEServer* pServer) {
bleConnected = false; pServer->startAdvertising(); }
};
```

```
class MyCharacteristicCallbacks : public
BLECharacteristicCallbacks {
void onWrite(BLECharacteristic* pCharacteristic) {
String value = pCharacteristic->getValue().c_str();
JsonDocument doc;
deserializeJson(doc, value);
String cmd = doc["cmd"].as<String>();

if (cmd == "pump_on") {
digitalWrite(PUMP_PIN, HIGH);
pumpState = true;
pCharacteristic->setValue("{\"status\":\"on\"}");
pCharacteristic->notify();
}
else if (cmd == "pump_off") {
digitalWrite(PUMP_PIN, LOW);
pumpState = false;
pCharacteristic->setValue("{\"status\":\"off\"}");
pCharacteristic->notify();
}
else if (cmd == "status") {
float temp     = dht.readTemperature();
float humidity = dht.readHumidity();
bool dhtOk     = !isnan(temp) && !isnan(humidity);
if (!dhtOk) { temp = 0; humidity = 0; }
int soil       = analogRead(SOIL_PIN);
bool soilOk    = soil > 0 && soil < 4095;
int soilPercent = soilOk ? map(soil, 4095, 0, 0, 100) : 0;
String json = "{";
json += "\"device\":\"" + deviceName + "\",";
json += "\"pump\":"     + String(pumpState ? 1 : 0) +
",";
json += "\"soil\":"     + String(soilPercent) + ",";
json += "\"soilConnected\":" + String(soilOk ? "true"
: "false") + ",";
json += "\"temp\":"     + String(temp, 1) + ",";
json += "\"humidity\":" + String(humidity, 1) + ",";
json += "\"dhtConnected\":" + String(dhtOk ? "true"
```

```
: "false") + ",";
json += "\"waterCount\":" + String(waterCount) +
",";
json += "\"totalWaterTime\":" +
String(totalWaterTime);
json += "}";
pCharacteristic->setValue(json.c_str());
pCharacteristic->notify();
}
else if (cmd == "auto") {
float humidity = dht.readHumidity();
float temp     = dht.readTemperature();
if (isnan(humidity) || isnan(temp)) {
pCharacteristic-
>setValue("{\"status\":\"sensor_error\"}");
pCharacteristic->notify();
return;
}
if (humidity < 40) {
digitalWrite(PUMP_PIN, HIGH);
pumpState = true;
delay(5000);
digitalWrite(PUMP_PIN, LOW);
pumpState = false;
waterCount++;
totalWaterTime += 5;
prefs.putInt("waterCount", waterCount);
prefs.putFloat("totalWater", totalWaterTime);
pCharacteristic->setValue("{\"status\":\"watered\"}");
} else {
pCharacteristic->setValue("{\"status\":\"notneed\"}");
}
pCharacteristic->notify();
}
}
};

void addCORS() {
```

```
server.sendHeader("Access-Control-Allow-Origin",
"*");
server.sendHeader("Access-Control-Allow-Methods",
"GET, POST");
server.sendHeader("Access-Control-Allow-Headers",
"Content-Type, X-Secret-Key");
}

bool checkAuth() {
if (secretKey == "") return true;
return server.header("X-Secret-Key") == secretKey;
}

void handleDiscover() {
addCORS();
String json = "{";
json += "\"device\":\"" + deviceName + "\",";
json += "\"type\":\"SmartDrop\",";
json += "\"version\":\"1.0\",";
json += "\"configured\":" + String(configured ? "true"
: "false");
json += "}";
server.send(200, "application/json", json);
}

void handleStatus() {
addCORS();
if (!checkAuth()) { server.send(401, "application/json",
"{\"error\":\"Unauthorized\"}"); return; }
float temp     = dht.readTemperature();
float humidity = dht.readHumidity();
bool dhtOk     = !isnan(temp) && !isnan(humidity);
if (!dhtOk) { temp = 0; humidity = 0; }
int soil       = analogRead(SOIL_PIN);
bool soilOk    = soil > 0 && soil < 4095;
int soilPercent = soilOk ? map(soil, 4095, 0, 0, 100) : 0;
String json = "{";
json += "\"device\":\"" + deviceName + "\",";
json += "\"pump\":"     + String(pumpState ? 1 : 0) +
```

```
";";
json += "\"pumpConnected\":true,";
json += "\"soil\":"     + String(soilPercent) + ",";
json += "\"soilConnected\":" + String(soilOk ? "true"
: "false") + ",";
json += "\"temp\":"     + String(temp, 1) + ",";
json += "\"humidity\":" + String(humidity, 1) + ",";
json += "\"dhtConnected\":" + String(dhtOk ? "true"
: "false") + ",";
json += "\"waterCount\":" + String(waterCount) +
",";
json += "\"totalWaterTime\":" +
String(totalWaterTime);
json += "}";
server.send(200, "application/json", json);
}

void handlePumpOn() {
addCORS();
if (!checkAuth()) { server.send(401, "application/json",
"{\"error\":\"Unauthorized\"}"); return; }
digitalWrite(PUMP_PIN, HIGH);
pumpState = true;
prefs.putBool("pump", true);
server.send(200, "application/json",
"{\"status\":\"on\"}");
}

void handlePumpOff() {
addCORS();
if (!checkAuth()) { server.send(401, "application/json",
"{\"error\":\"Unauthorized\"}"); return; }
digitalWrite(PUMP_PIN, LOW);
pumpState = false;
prefs.putBool("pump", false);
server.send(200, "application/json",
"{\"status\":\"off\"}");
}
```

```
void handleAutoWater() {
addCORS();
if (!checkAuth()) { server.send(401, "application/json",
"{\"error\":\"Unauthorized\"}"); return; }
float humidity = dht.readHumidity();
float temp     = dht.readTemperature();
if (isnan(humidity) || isnan(temp)) {
server.send(200, "application/json",
"{\"status\":\"sensor_error\"}");
return;
}
if (humidity < 40) {
digitalWrite(PUMP_PIN, HIGH);
pumpState = true;
delay(5000);
digitalWrite(PUMP_PIN, LOW);
pumpState = false;
waterCount++;
totalWaterTime += 5;
prefs.putInt("waterCount", waterCount);
prefs.putFloat("totalWater", totalWaterTime);
server.send(200, "application/json",
"{\"status\":\"watered\"}");
} else {
server.send(200, "application/json",
"{\"status\":\"notneed\"}");
}
}

void handleRename() {
addCORS();
if (!checkAuth()) { server.send(401, "application/json",
"{\"error\":\"Unauthorized\"}"); return; }
String body = server.arg("plain");
JsonDocument doc;
deserializeJson(doc, body);
deviceName = doc["name"].as<String>();
prefs.putString("deviceName", deviceName);
server.send(200, "application/json",
```

```
"{\"status\":\"ok\"}");
}

void handleReset() {
addCORS();
if (!checkAuth()) { server.send(401, "application/json",
"{\"error\":\"Unauthorized\"}"); return; }
waterCount = 0; totalWaterTime = 0;
prefs.putInt("waterCount", 0);
prefs.putFloat("totalWater", 0);
server.send(200, "application/json",
"{\"status\":\"reset\"}");
}

void handleSetup() {
addCORS();
if (server.method() != HTTP_POST) { server.send(405,
"application/json", "{\"error\":\"Method not
allowed\"}"); return; }
String body = server.arg("plain");
JsonDocument doc;
deserializeJson(doc, body);
wifiSSID   = doc["ssid"].as<String>();
wifiPass   = doc["password"].as<String>();
secretKey  = doc["secretKey"].as<String>();
deviceName = doc["deviceName"].as<String>();
prefs.putString("ssid", wifiSSID);
prefs.putString("pass", wifiPass);
prefs.putString("secretKey", secretKey);
prefs.putString("deviceName", deviceName);
prefs.putBool("configured", true);
server.send(200, "application/json",
"{\"status\":\"ok\"}");
delay(1000);
ESP.restart();
}

void handleFactoryReset() {
addCORS();
```

```
if (!checkAuth()) { server.send(401, "application/json",
"{\"error\":\"Unauthorized\"}"); return; }
prefs.clear();
server.send(200, "application/json",
"{\"status\":\"reset\"}");
delay(1000);
ESP.restart();
}

void startAPMode() {
WiFi.softAP("SmartDrop-Setup");
Serial.println("AP Mode: SmartDrop-Setup");
Serial.println("IP: " + WiFi.softAPIP().toString());
}

void initBLE() {
BLEDevice::init(deviceName.c_str());
pServer = BLEDevice::createServer();
pServer->setCallbacks(new MyServerCallbacks());
BLEService* pService = pServer-
>createService(SERVICE_UUID);
pCharacteristic = pService->createCharacteristic(
CHARACTERISTIC_UUID,
BLECharacteristic::PROPERTY_READ |
BLECharacteristic::PROPERTY_WRITE |
BLECharacteristic::PROPERTY_NOTIFY
);
pCharacteristic->addDescriptor(new BLE2902());
pCharacteristic->setCallbacks(new
MyCharacteristicCallbacks());
pService->start();
BLEAdvertising* pAdvertising =
BLEDevice::getAdvertising();
pAdvertising->addServiceUUID(SERVICE_UUID);
pAdvertising->setScanResponse(true);
BLEDevice::startAdvertising();
}

void setup() {
```

```
Serial.begin(115200);
pinMode(PUMP_PIN, OUTPUT);
digitalWrite(PUMP_PIN, LOW);
dht.begin();

prefs.begin("smartdrop", false);
configured     = prefs.getBool("configured", false);
wifiSSID       = prefs.getString("ssid", "");
wifiPass       = prefs.getString("pass", "");
secretKey      = prefs.getString("secretKey", "");
deviceName     = prefs.getString("deviceName",
"SmartDrop-1");
waterCount     = prefs.getInt("waterCount", 0);
totalWaterTime = prefs.getFloat("totalWater", 0);

if (configured && wifiSSID != "") {
WiFi.begin(wifiSSID.c_str(), wifiPass.c_str());
int tries = 0;
while (WiFi.status() != WL_CONNECTED && tries <
20) {
delay(500); tries++;
}
if (WiFi.status() == WL_CONNECTED) {
MDNS.begin("smartdrop");
MDNS.addService("http", "tcp", 80);
} else {
startAPMode();
}
} else {
startAPMode();
}

bool lastPump = prefs.getBool("pump", false);
if (lastPump) { digitalWrite(PUMP_PIN, HIGH);
pumpState = true; }

server.on("/discover",      handleDiscover);
server.on("/status",        handleStatus);
```

```
server.on("/pump/on",       handlePumpOn);
server.on("/pump/off",      handlePumpOff);
server.on("/auto",          handleAutoWater);
server.on("/reset",         handleReset);
server.on("/rename",        handleRename);
server.on("/setup",         handleSetup);
server.on("/factory-reset", handleFactoryReset);
server.begin();

initBLE();
}

void loop() {
server.handleClient();

if (millis() - lastAutoCheck >= 10000) {
lastAutoCheck = millis();

int soil    = analogRead(SOIL_PIN);
bool soilOk = soil > 0 && soil < 4095;

if (!soilOk) return;

int soilPercent = map(soil, 4095, 0, 0, 100);

float temp     = dht.readTemperature();
float humidity = dht.readHumidity();
sendToThingSpeak(temp, humidity, soilPercent,
pumpState);

if (soilPercent < 40) {
digitalWrite(PUMP_PIN, HIGH);
pumpState = true;
delay(5000);
digitalWrite(PUMP_PIN, LOW);
pumpState = false;
waterCount++;
totalWaterTime += 5;
```

```
            prefs.putInt("waterCount", waterCount);
            prefs.putFloat("totalWater", totalWaterTime);
            }
            }
}
```

Conclusion

In conclusion, the developed smart irrigation system using an ESP32 successfully achieves automated water management based on real-time soil moisture monitoring. The system efficiently controls the irrigation process by turning the water pump ON and OFF according to environmental conditions.

The integration of a soil moisture sensor and a DHT11 provides accurate environmental data, while the IoT features such as WiFi and Bluetooth (BLE) enable remote monitoring and control through a mobile application and web interface.

Overall, the project demonstrates a low-cost, reliable, and scalable solution for smart agriculture. It reduces water wastage, minimizes manual intervention, and improves irrigation efficiency. Future improvements may include advanced AI-based irrigation decisions and cloud data storage.

References

1. Espressif Systems. *ESP32 Technical Reference Manual*. Available at: https://www.espressif.com
2. Arduino. *Arduino IDE Documentation*. Available at: https://www.arduino.cc
3. Espressif Systems. *ESP32 WiFi & BLE Programming Guide*. Available at: https://docs.espressif.com
4. Adafruit Industries. *DHT11 Temperature and Humidity Sensor Guide*. Available at: https://learn.adafruit.com
5. Standard Electronics Notes. *Soil Moisture Sensor Working Principle*.